Chương 2

TRÍ TUỆ NHÂN TẠO VÀ MÁY HỌC

Giới hạn AI vào các mô hình giải quyết vấn đề dựa trên tri thức, kinh nghiệm tích lũy được từ dữ liệu quan trắc (observations), là những mô hình được học từ dữ liệu, gọi là các mô hình máy học, hay đơn giản là máy học (machine learning).

Không mất tính tổng quát, dữ liệu được cho là các véc-tơ số hóa

$\Omega = \{(\boldsymbol{x}^{(i)}, y^{(i)}) \in D \times R\}_i^N$,

trong đó $\boldsymbol{x}^{(i)} = (x_1^{(i)}, \dots, x_n^{(i)}) \in D \subset \mathbb{R}^n$ là giá trị của các biến độc lập (independent variables), và $y^{(i)} \in R$ là giá trị ($R \subset \mathbb{R}$), hay nhãn ($R \subset \mathbb{N}$) của biến phụ thuộc (dependent variable) của mẫu thứ ith (sample) quan sát được. Bài toán của ta là tìm mô hình, là một hàm $f: D \to R$ để mô hình hóa các dữ liệu quan sát được sao

$f(\boldsymbol{x}) = y, \forall(\boldsymbol{x}, y) \in \Omega$ (*).

Hơn nữa, mô hình còn phải có khả năng tổng quát hóa, nghĩa là nó có khả năng tính chính xác được

$f(\boldsymbol{x})$, với $(\boldsymbol{x}, f(\boldsymbol{x})) \notin \Omega$ (**).

THIẾT LẬP BÀI TOÁN TỐI ƯU

Mô hình hóa là bài toán tìm kiếm

Nhiệm vụ tìm hàm f là bài toán khó vì nó "mông lung" vì ngoài tiêu chí (*) ta không có bất kỳ dấu hiệu nào về hàm f. Để nhiệm vụ "cụ thể" hơn, thay vì tìm hàm f bất kỳ, ta tìm hàm f có một dạng nào đó. Dạng hàm đó được thể hiện qua tập các tham số $\boldsymbol{w} \in \mathbb{R}^m$, và ta ký hiệu $f_{\boldsymbol{w}}$ là hàm cần tìm. Bằng cách giới hạn không gian tìm kiếm vào không gian các hàm tham số, bài toán tìm kiếm hàm được đưa về tìm kiếm véc-tơ, hay phổ biến hơn, là các ma-trận số, ta sẽ gọi chung là tập tham số (parameters). Hơn nữa, thay vì tìm theo tiêu chí (*), ta sẽ parameters $\boldsymbol{w}$ với tiêu chí (*) được nới lỏng thành

$f_{\boldsymbol{w}}(\boldsymbol{x}) = y \pm \varepsilon$ hay $|f_{\boldsymbol{w}}(\boldsymbol{x}) - y| \leq \varepsilon, \forall(\boldsymbol{x}, y) \in \Omega$ (***),

Với ε là hằng dương cho trước, và được gọi là sai số.

Sai số ε được gọi là sai số của mô hình nếu

$\frac{1}{N}\sum_{i=1}^{N}|f_{\boldsymbol{w}}(x^{(i)}) - y^{(i)}| \leq \varepsilon$ (#).

Mô hình hóa là bài toán tơi ưu

Với tiêu chí tìm kiếm (#) mới thiết lập, giải bài toán mô hình hóa giờ có thể giải bằng các phương pháp tối ưu của AI mà ta đã biết.

Đặt

$\mathcal{L}_\Omega(\boldsymbol{w}) = \frac{1}{N}\Sigma_{i=1}^{N}|f_{\boldsymbol{w}}(x^{(i)}) - y^{(i)}|$,

với $N = |\Omega|$ là số mẫu quan sát được, và ta ký hiệu $\mathcal{L}(\boldsymbol{w})$ thay cho $\mathcal{L}_\Omega(\boldsymbol{w})$. $\mathcal{L}$ được gọi là hàm mất mát (loss function) trong bài toán mô hình hóa.

Bài toán mô hình hóa tập dữ liệu Ω được đưa về bài toán tối ưu

$\boldsymbol{w}^* = \min_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w})$.

Lồi hóa hàm mất mát

Sử dụng các ký hiệu của đại số ma-trận, biểu diễn lại Ω thành 2 ma-trận $A = \begin{bmatrix} \tilde{\boldsymbol{x}}^{(1)} \\ \vdots \\ \tilde{\boldsymbol{x}}^{(N)} \end{bmatrix} \in \mathbb{R}^{N\times\tilde{n}}$ là kết quả ghép chồng các biến độc lập $\tilde{\boldsymbol{x}}$ của mẫu $(\boldsymbol{x}, y) \in \Omega$, với $\tilde{\boldsymbol{x}}$ là biến đổi của $\boldsymbol{x}$ theo một biến đổi xác định T nào đó, $\tilde{\boldsymbol{x}} = T(\boldsymbol{x}) \in \mathbb{R}^{\tilde{n}}$; và $B = \begin{bmatrix} y^1 \\ \vdots \\ y^N \end{bmatrix} \in \mathbb{R}^N$là ghép chồng các biến phụ thuộc tương ứng.

Không mất tính tổng quát, ta giả sử T là phép biên đổi đơn vị, $T(\boldsymbol{x}) = Identity(\boldsymbol{x}) = \boldsymbol{x}$. Như vậy $\tilde{n} = n$. Và ta có thể xem hàm mất mát được định nghĩa như

$\mathcal{L}(\boldsymbol{w}) = ||A \cdot \boldsymbol{w} - B||^2$,

với $A \in \mathbb{R}^{N\times n}, B \in \mathbb{R}^N$ và giả sử N rất lớn hơn n, $N \gg n$.

Mệnh đề (tối ưu hàm mất mát). Nếu $Rank(A) = n$. Nghiệm tối ưu của bài toán tối ưu

$\boldsymbol{w}^* = \min_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}) = \min_{\boldsymbol{w}} ||A \cdot \boldsymbol{w} - B||^2$,

có dạng

$\boldsymbol{w}^* = (A^T \cdot A)^{-1} \cdot A^T \cdot B$.

Chứng minh.

Trước tiên, ta chưng minh hàm $\mathcal{L}(\boldsymbol{w}) = ||A \cdot \boldsymbol{w} - B||^2$ là hàm lồi.

Thực vậy,

$\mathcal{L}(\boldsymbol{w}) = \left|\left|A \cdot \boldsymbol{w} - B\right|\right|^2 = < A \cdot \boldsymbol{w} - B, A \cdot \boldsymbol{w} - B >$

$= (A \cdot \boldsymbol{w} - B)^T \cdot (A \cdot \boldsymbol{w} - B)$

$= ((A \cdot \boldsymbol{w})^T - B^T) \cdot (A \cdot \boldsymbol{w} - B)$

$= (\boldsymbol{w}^T \cdot A^T - B^T) \cdot (A \cdot \boldsymbol{w} - B)$

$= \boldsymbol{w}^T \cdot A^T \cdot A \cdot \boldsymbol{w} - B^T \cdot A \cdot \boldsymbol{w} - \boldsymbol{w}^T \cdot A^T \cdot B + B^T \cdot B$

$= \boldsymbol{w}^T \cdot (A^T \cdot A) \cdot \boldsymbol{w} - 2(B^T \cdot A) \cdot \boldsymbol{w} + B^T \cdot B.$

Suy ra, vec-tơ đạo hàm

$\nabla\mathcal{L}(\boldsymbol{w}) = 2(A^T \cdot A) \cdot \boldsymbol{w} - 2(B^T \cdot A).$

Và ma-trận Hesse

$\nabla^2\mathcal{L}(\boldsymbol{w}) = 2(A^T \cdot A).$

Với véc-tơ $\boldsymbol{h} \in \mathbb{R}^n$ bất kỳ,

$\boldsymbol{h}^T \cdot \nabla^2\mathcal{L}(\boldsymbol{w}) \cdot \boldsymbol{h} = 2(\boldsymbol{h}^T \cdot A^T \cdot A \cdot \boldsymbol{h})$

$= 2 < A \cdot \boldsymbol{h}, A \cdot \boldsymbol{h} >$

$= 2||A \cdot \boldsymbol{h}|| \geq 0.$

Theo định nghĩa, ma-trận Hesse $\nabla^2\mathcal{L}(\boldsymbol{w})$ nửa xác định dương. Áp dụng định lý tối ưu lồi trong chương trước $\mathcal{L}(\boldsymbol{w})$ lồi, và lời giải tối ưu là nghiệm phương trình

$\nabla\mathcal{L}(\boldsymbol{w}^*) = \mathbf{0}$

$\Leftrightarrow\ 2(A^T \cdot A) \cdot \boldsymbol{w}^* - 2(B^T \cdot A) = \mathbf{0}$

$\Leftrightarrow (A^T \cdot A) \cdot \boldsymbol{w}^* = (B^T \cdot A)$ #Do $Rank(A) = n$, nên $A^T \cdot A$ khả nghịch.

$\Leftrightarrow \boldsymbol{w}^* = (A^T \cdot A)^{-1} \cdot A^T \cdot B.$

CÁC PHƯƠNG PHÁP THỐNG KÊ

Hồi quy tuyến tính

Hồi quy tuyến tính (linear regression) là phương pháp thống kê truyền thống có thể được sử dụng để giải bài toán mô hình hóa. Trong hồi quy tuyến tính, mô hình được giả sử là hàm $f_{\boldsymbol{w}}: \mathbb{R}^n \to \mathbb{R}$, với $w \in \mathbb{R}^n$, có dạng

$f_{\boldsymbol{w}}(\boldsymbol{x}) = f_{\boldsymbol{w}}(x_1, \dots, x_n) = w_1 x_1 + \cdots + w_n x_n.$

Bằng cách đặt

$$A = \begin{bmatrix} \boldsymbol{x}^{(1)} \\ \vdots \\ \boldsymbol{x}^{(N)} \end{bmatrix} \in \mathbb{R}^{N \times n}, B = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(N)} \end{bmatrix} \in \mathbb{R}^N,$$

và hàm mất mát

$$\mathcal{L}(\boldsymbol{w}) = ||A \cdot \boldsymbol{w} - B||^2,$$

được xây dựng như trong phần trước. Thì tham số

$$w = (A^T \cdot A)^{-1} \cdot A^T \cdot B,$$

như được chứng minh trong phần trước là nghiệm tối ưu của bài toán

$$\boldsymbol{w}^* = \min_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}) = \min_{\boldsymbol{w}} ||A \cdot \boldsymbol{w} - B||^2.$$

Lưu ý hàm hồi quy tuyến tính $f_{\boldsymbol{w}}(\boldsymbol{x}) = f_{\boldsymbol{w}}(x_1, \dots, x_n) = w_1 x_1 + \cdots + w_n x_n$, với $\boldsymbol{w}$ được xác định như trên không đi qua gốc $\mathbf{O}$. Trong trường hợp muốn cho hàm hồi quy qua gốc $\mathbf{O}$, hàm $f_{\boldsymbol{w}}$ có dạng

$$f_{\boldsymbol{w}}(x_1, \dots, x_n) = w_0 + \Sigma_{i=1}^{n} w_i x_i.$$

Để thực hiện, ta mở rộng $\boldsymbol{w} \in \mathbb{R}^n$ thành $\widetilde{\boldsymbol{w}} = (w_0, \boldsymbol{w}) \in \mathbb{R}^{n+1}$, và $\widetilde{\boldsymbol{x}} = T(\boldsymbol{x}) = (1, \boldsymbol{x}) \in \mathbb{R}^{n+1}$. Và ta có thể xác định $\widetilde{\boldsymbol{w}}^* = (A^T \cdot A)^{-1} \cdot A^T \cdot B$, với A, B được xác định như trước, chỉ khác thay $\boldsymbol{x}$ bằng $\widetilde{\boldsymbol{x}}$ vào các hàng tương ứng của A.

Thử dữ liệu

Hồi quy tuyến tính như trình bày trên sẽ có sai số lớn nếu dữ liệu không có quan hệ tuyến tính giữa các biến độc lập và biến phụ thuộc. Để cải thiện hiệu quả của mô hình, nghĩa là giảm sai số mô hình, phân tích dữ liệu được thực hiện để tuyến tính hóa dữ liệu, bằng cách thử thực hiện một số biến đổi trên dữ liệu, đặc biệt dữ liệu độc lập, và sử dụng phương pháp hồi quy tuyến tính trên để thử lại với tập dữ liệu đã biến đổi. Phương pháp thử dữ liệu này, cũng như phương pháp hồi quy tuyến tính, thực chất là phương pháp sử dụng kỹ thuật bình phương tối tiểu và gọi là chúng là bình phương tối tiểu phổ biến (OLS – Ordinary Least Squares). Quy trình (workflow) gồm các bước chính sau:

(1) Xác định tập p hàm cơ sở dựa trên quá trình phân tích dữ liệu gốc, $f_p: \mathbb{R}^n \to \mathbb{R}$.
(2) Sử dụng phép biến đổi $\widetilde{\boldsymbol{x}} = T(\boldsymbol{x}) = (f_1(\boldsymbol{x}), \dots, f_p(\boldsymbol{x})), \forall (\boldsymbol{x}, f(\boldsymbol{x})) \in \Omega$.

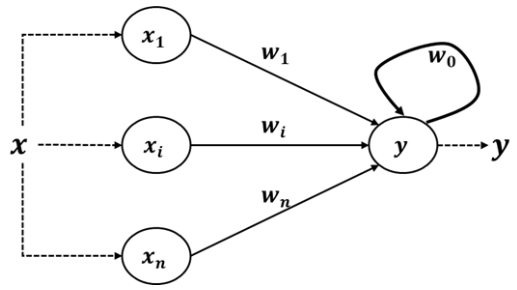
(3) Thực hiện hồi quy tuyến tính trên tập dữ liệu đã biến đổi, tìm tham số $\boldsymbol{w}$ của mô hình $f_{\boldsymbol{w}}$.
(4) Tính sai số của mô hình.

Bước khó nhất trong quy trình trên là chọn tập các hàm cơ sở $\{f_1, \dots, f_p\}$ để xây dựng phép biến đổi $T(\boldsymbol{x}) = (f_1(\boldsymbol{x}), \dots, f_p(\boldsymbol{x}))$, nhằm tuyến tính hóa dữ liệu. Việc chọn bao nhiêu hàm cơ sở, p, và hàm cơ sở $f_i, i = 1, \dots, p$ như thế nào phụ thuộc vào tri thức người phân tích dữ liệu, và tri thức về miền ứng dụng.

Trong trường hợp người dùng không phải chuyên gia trong lãnh vực ứng dụng, cũng như ít kinh nghiệm trong phân tích dữ liệu, các phương pháp máy học trong trí tuệ nhân tạo có thể được thử nghiệm. Tài liệu này giới hạn vào các mô hình mạng nơ-ron (neural networks) và sử dụng nó để phát triển các mô hình AI, gọi là ML-AI (Machine Learning-Artificial Intelligence) hay mạng nơ-ron nhân tạo (ANN – Artificial Neural Network).

MẠNG NƠ-RON

Bằng cách biểu diễn mô hình hồi quy tuyến tính $f_{\boldsymbol{w}}(x) = f_{\boldsymbol{w}}(x_1, \dots, x_n) = w_0 + w_1 x_1 + \dots + w_n x_n$ như hình vẽ, ta có một minh họa trực quan về khái niệm mạng nơ-ron nhân tạo ANN, và từ nay ta sẽ gọi đơn giản là mạng nơ-ron (NN – Neural Network).



Hình vẽ trên minh họa một mạng nơ-ron 2 tầng (2-layered neural network). Mỗi nút (node) được gọi là một nơ-ron, và cung nối giữa 2 nút được gắn một giá trị $w \in \mathbb{R}$, gọi là cường độ hay trọng số. Mỗi nút, hoặc nhận dữ liệu trực tiếp từ dữ liệu đầu vào, gọi là nút nhập hay nơ-ron đầu vào (input neuron); hoặc nhận dữ liệu từ các nút kề, với cường độ kích thích của cung tương ứng. Nút trả kết quả tính toán của mạng nơ-ron gọi là nút xuất (output neuron).

Mạng nơ-ron được sử dụng để xây dựng các mô hình không tuyến tính (non-linear) do dữ liệu có quan hệ quá phức tạp, khó tuyến tính hóa được. Mạng nơ-ron có thể được xây dựng cho bài toán nhận dạng (identification), nhận đầu vào là véc-tơ đặc trưng (feature vector) của một đối tượng cần nhận dạng và đầu ra là true (1) hay false (0); hay cho bài toán phân loại (classification) nhận vào véc-tơ đặc trưng và trả ra đối tượng thước lớp có nhãn nào

trong các nhãn $C = \{c_1, \dots, c_k\}$; hay cho bài toán dự báo như các mô hình hồi quy tuyến tính trên, nghĩa là cho ra một giá trị liên tục từ các giá trị đặc trưng quan sát được. Trong thực tế, đầu ra của các dữ liệu dự báo cũng thường được rời rạc hóa (discretization).

Mạng nhận thức

Mạng nhận thức (perceptron neural network or perceptron) có thể xem như một sự mô phỏng hồi quy tuyến tính cho bài toán ra quyết định (make a decision) đơn giản là đúng (true) hay sai (false), như bài toán định danh chẳng hạn, theo đó, một perceptron là một 2-layered NN như hình vẽ, với chỉ 1 nút đầu ra nhận giá trị thuộc {0, 1}, và chỉ có cung nối giữa 1 nút của lớp nhập (input layer) với nút đầu ra.

Khái niệm và hoạt động

Có thể hình dung một perceptron như một hệ thống gồm 5 nơ-ron cảm giác trên 5 đầu ngón tay. Khi vô tình chạm vào bình nước nóng, nhiệt độ của bình chuyển thành tín hiệu x_i vào cho 5 nơ-ron giác quan ở đầu ngón tay. Các tín hiệu này được phóng đại tương ứng với độ nhạy w_i của nơ-ron và truyền tín hiệu kích thích với cường độ $w_i x_i$ đến nơ-ron thần kinh trung ương để tổng hợp thành y. Tùy thuộc cường độ tổng hợp có lớn hơn ngưỡng kích hoạt w_0 (bias) hay không mà hệ thống quyết định rụt tay lại hay không.

Với đặc trưng đầu vào là một véc-tơ *n*-chiều, $\boldsymbol{x} \in D \subset \mathbb{R}^n$, một perceptron được mô hình hóa bằng một véc-tơ trọng số $\boldsymbol{w} \in W \subset \mathbb{R}^{n+1}$. Nhận vào véc-tơ đầu vào $\boldsymbol{x} = (x_1, \dots, x_n)$, giá trị đầu ra y được lượng giá qua hàm F sau.

$F_{\boldsymbol{w}}(x)$:

$t = 0$

$For\ (i = 1\ to\ n): t = t + w_i * x_i$

$If\ (t > w_0): y = 1\ else: y = 0.$

Trong thủ tục Forrward, tham số mô hình $\boldsymbol{w}$ được xác định trước trong tiến trình học (learng) để xây dựng perceptron từ dữ liệu quan trắc (observation dataset) Ω.

Hàm mức

Tín hiệu đầu ra của mạng perceptron chỉ có thể lượng giá được qua hàm $F_{\boldsymbol{w}}$ nếu tham số $\boldsymbol{w}$ được cho. Có thể sử dụng phương pháp tìm kiếm leo-đồi (hill clibing) trong chương trước và cài đặt thuật giải giảm đạo hàm hàm mất mát $\nabla\mathcal{L}(\boldsymbol{w})$ để xây dựng tham số $\boldsymbol{w}$ cho perceptron, với $\mathcal{L}$ được xác định như:

$\mathcal{L}(\boldsymbol{w}) = \frac{1}{2N}\Sigma_{i=1}^{N}\left(F_{\boldsymbol{w}}(x^{(i)}) - y^{(i)}\right)^2.$

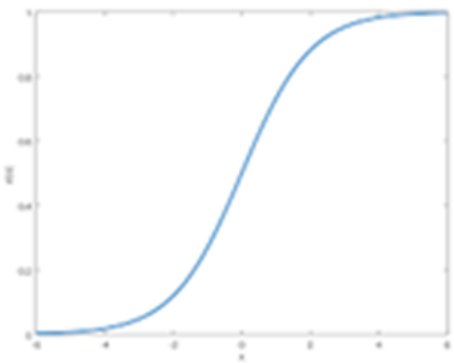
Khó khăn nằm ở chỗ $F_{\boldsymbol{w}}(\boldsymbol{x})$ chỉ nhận 1 trong 2 giá trị là 0 hay 1, và như thế véc-tơ đạo hàm $\nabla\mathcal{L}(\boldsymbol{w})$ không thể tính.

Để vượt qua khó khan trong việc tính $\nabla\mathcal{L}(\boldsymbol{w})$, bây giờ, thay vì sử dụng hàm dấu

$sign_{\theta}(u) = \begin{cases} 0, & u < \theta \\ 1, & u \geq \theta \end{cases},$

ta sẽ sử dụng một hàm liên tục để có thể tính được véc-tơ đạo hàm hàm mất mát.

Trong máy học mạng nơ-ron, bao gồm cả perceptron, hàm mức có dạng chữ "S" (sigmoid function) như minh họa trong hình vẽ, là hàm thường được sử dụng, và được định nghĩa hình thức như sau.



Định nghĩa (sigmoid function). Hàm $s: \mathbb{R} \to \mathbb{R}$ là hàm có những tính chất sau:

- Liên tục (continuity),
- Bị chặn (bounded), và
- Trơn (smoothness).

Trong các hàm sigmoid, hàm logistic có nguồn gốc từ thống kê là hàm được dùng phổ biến nhất, nên phương pháp perceptron còn được gọi là hồi quy logistic (logistic regression).

Định nghĩa (logistic function). Hàm $g: \mathbb{R} \to [0, 1]$ là hàm logistic được xác định như

$g(u) = \frac{1}{1+e^{-u}}.$

Lưu ý là đạo hàm $g'(u)$ hàm logistic được tính dễ dàng từ các giá trị của hàm $g(u)$ tại u. Thực vậy,

$g'(u) = \frac{dg}{du}(u) = g(u)g((u) - 1).$

Sử dụng hàm logistics, hàm F lượng giá kết quả của nơ-ron đầu ra ở trên được sửa lại thành

$Forward(\boldsymbol{x})$:

$u = w_0$

$For\ (i = 1\ to\ n): u = u + w_i * x_i$

$Return\ g(u)$.

Máy học perceptron

Đến đây, ta đã sẵn sàng để xây dựng phương pháp máy học, học từ tập mẫu

$\Omega = \left\{(\boldsymbol{x}^i, y^i) = (x_1^{(i)}, \dots, x_n^{(i)}, y^{(i)}) \in D \times R\right\}_{i=1}^{N}$,

với $D \subset \mathbb{R}^n, R \subset \mathbb{R}$.

Trong các tài liệu về máy học-mạng nơ-ron, thuật giải sau được gọi thuật giải lan truyền ngược lỗi (error back-propagation hay back-propagation), và gồm 2 tiến trình đan xen: lan truyền tiến (Forward) để tính trung bình lỗi, $E_{\boldsymbol{w}} = \mathcal{L}(\boldsymbol{w})$ của perceptron trên tập huấn luyện (training set) Ω, với $\boldsymbol{w}$ là tham số mô hình tại bước lặp hiện hành; và tiến trình lan truyền ngược lỗi (Back) để cập tính véc-tơ đạo hàm hàm lỗi $\nabla\mathcal{L}(\boldsymbol{w})$, và cập nhật tham số $\boldsymbol{w} = \boldsymbol{w} + \alpha\nabla\mathcal{L}(\boldsymbol{w})$, với $\alpha \in (0, 1)$ là hệ số học.

Giả sử $z = Forward(\boldsymbol{x}) = g(u)$, $u = w_0 + w_1x_1 + \cdots + w_nx_n$, công thức tính đạo hàm thành phần $\frac{\partial\mathcal{L}(w)}{\partial w_i}, i = 0, 1, \dots, n$, của véc-tơ đạo hàm $\nabla\mathcal{L}(\boldsymbol{w})$ được thiết lập như sau.

Theo quy tắc chuỗi đạo hàm (ta bỏ đi các tham số cho công thức đơn giản),

$\frac{\partial\mathcal{L}}{\partial w_i} = \frac{\partial\mathcal{L}}{\partial z}\frac{\partial z}{\partial u}\frac{\partial u}{\partial w_i}$ (*).

Và từng thành phần của (*) được tính như sau.

- $\frac{\partial\mathcal{L}}{\partial z}$

Do $\mathcal{L} = \frac{1}{2}\ (z - t)^2$, nên

$\frac{\partial\mathcal{L}}{\partial z} = z - y$.

- $\frac{\partial z}{\partial u}$

Do $z = g(u)$, nên

$\frac{\partial z}{\partial u} = g'(u) = g(u)(1 - g(u)) = z(1 - z).$

- $\frac{\partial u}{\partial w_i}$

Do $u = w_0 + w_1 x_1 + \cdots + w_n x_n$, nên

$\frac{\partial u}{\partial w_i} = \begin{cases} 1, & i = 0 \\ x_i, & i \neq 0 \end{cases}.$

Vậy $(*) \Leftrightarrow \frac{\partial \mathcal{L}}{\partial w_i} = \begin{cases} (z - y)z(1 - z), & i = 0 \\ (z - y)z(1 - z)x_i, & i \neq 0 \end{cases}.$

Perception nhiều nút đầu ra

Trong trường hợp nơ-ron đầu ra thuộc tập dữ liệu rời rạc, $C = \{c_1, \dots, c_k\}$, như trong các bài toán phân loại, bằng cách biểu diễn các nhãn (label) đầu ra bằng dãy bit nhị phân thích hợp, perception bây giờ là hàm

$f_{\boldsymbol{w}}: D \to R, D \subset \mathbb{R}^n, R \subset \{0, 1\}^m,$

với đầu ra là véc-tơ nhị phân m-bit thay vì chỉ 1 bít như trước.

Có nhiều cách để mã hóa (encode) các nhãn đầu ra bằng chuỗi nhị phân m-bit, chẳng hạn,

- Nếu số lớp nhỏ, có thể chọn $m = k$, với quy ước $y_i = 1, y_{j \neq i} = 0, \forall j \neq i$, thì đối tượng cần phân loại thuộc lớp có nhãn c_i; hay nhãn c_i được biểu diễn bằng i bit liên tiếp có giá trị 1, các bit còn lại cho bằng 0.
- Trong trường hợp số lớp lớn, có thể biểu diễn kiểu phân tầng, chẳng hạn, với 2 tầng, dùng $\frac{m}{2}$ bit cho tầng cơ bản (segment), và $\frac{m}{2}$ bit còn lại cho tầng 2 (offset), như vậy có thể biểu diễn được tối đa $k = \frac{m}{2} \times \frac{m}{2} = \frac{m^2}{4}$ lớp; hay có thể dùng chính giá trị thập phân của chuỗi nhị phân để biểu diễn nhãn $c_i = c_{b_0 b_1 \dots b_{m-1}} = \Sigma_{j=0}^{m-1} b_j 2^j$.

Trong trượng hợp lớp nơ-ron đầu ra là một véc-tơ, tham số mô hình $\boldsymbol{w}$ sẽ là một ma-trận $w = [w_{ij}] \in \mathbb{R}^{(n+1) \times m}$, trong đó w_{ij} là cung nối giữa nơ-ron nhập $x_i, i = 1, 2, \dots, n$, với nơ-ron đầu ra $y_j, j = 1,2, \dots, m$; và w_{0j} là ngưỡng kích hoạt của nơ-ron đầu ra y_j. Đạo hàm hàm lỗi Δ_{ij} vẫn được tính theo cùng phương pháp như trên

$\Delta_{ij} = \frac{\partial \mathcal{L}}{\partial w_{ij}} = \begin{cases} (z - y)z(1 - z), & i = 0 \\ (z - y)z(1 - z)x_i, & i \neq 0 \end{cases}$, với mọi $j = 1, \dots, m$.

Hạn chế của perceptron

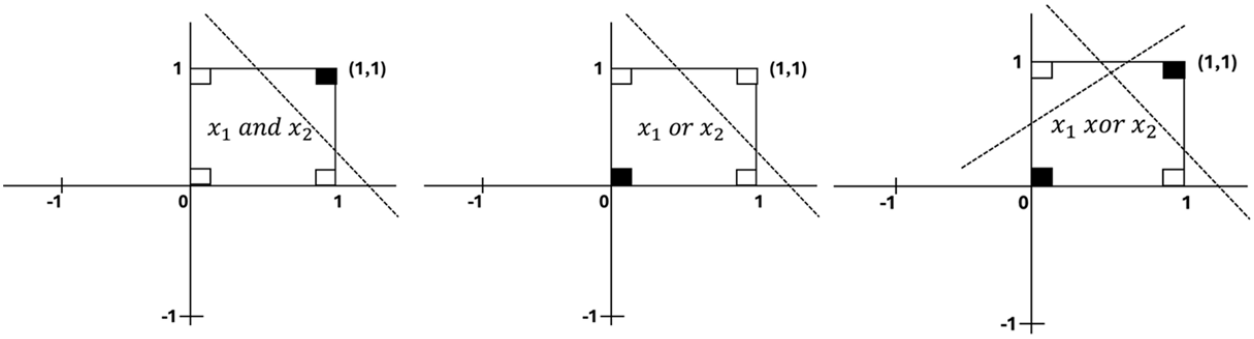
Perceptron là mô hình mạng nơ-ron nhân tạo tuy đơn giản nhưng đã thành công trong nhiều ứng dụng định danh cũng như phân loại. Tuy nhiên, trong trường hợp véc-tơ các thành phần độc lập không đủ thông tin để phân biệt các đối tượng, hoặc do số đặc trưng ít, hoặc do dữ liệu có bản chất không tuyến tính, hoặc do tồn tại nhiều nhiễu trong dữ liệu, … perceptron có thể không đạt được hiệu năng mong muốn.

Ví dụ. Giả sử cần xây dựng các perceptrons cho 3 bài toán logic

$proposition: \{0,1\} \times \{0,1\} \rightarrow \{0,1\}$.

Hình sau minh họa trực quan với $proposition = and, or$, và xor. Trong đó, màu của các hình vuông nhỏ thể hiện giá trị của phép tính mệnh đề

$$propossition(x_1, x_2) = \begin{cases} 0, & white \\ 1, & black \end{cases}.$$



Ta thấy, với 2 bài toán and và or, tồn tại siêu phẳng tách các hình vuông thành 2 vùng black and white thành các vùng tách biệt, vì 2 phép toán mệnh đề này là tuyến tính. Nhưng với bài toán không tuyến tính xor, không tồn tại siêu phẳng nào như thế.

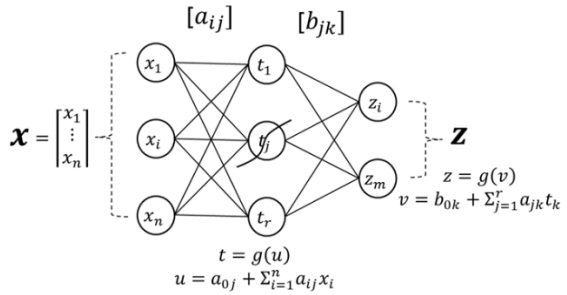
Có 2 giải pháp cho vấn đề hiệu năng của perceptron: hoặc thêm các lớp nơ-ron phụ, gọi là lớp ẩn (hidden layer), ngoài các nơ-ron nhập và nơ-ron xuất là cố định từ bài toán đặt ra, cấu trúc này được gọi là mạng đa lớp (multi-layered neural network); hoặc sử dụng các tri thức liên quan để phi-tuyến hóa dữ liệu đầu vào, chúng tôi gọi cấu trúc này là mạng nửa tĩnh (semi-static neural network).

Mạng đa lớp

Mạng 3-lớp truyền thẳng

Mạng đa lớp (multi-layered neural network), là một mạng gồm nhiều tầng (layer) nơ-ron, bao gồm tầng nhập (input layer), tầng ẩn, và 1 hay nhiều tầng ẩn.

Để đơn giản hóa, ta giới hạn mạng đa tầng là mạng truyền thẳng (feed forward), là một cấu trức, ở đó 1 nơ-ron ở lớp trước liên kết với mọi nơ-ron ở lớp sau, và chỉ các nơ-ron ở 2 lớp kề nhau mới có liên kết trực tiếp với nhau, như hình vẽ minh họa một mạng nơ-ron 3 lớp.



Cũng để đơn giản cho việc giải thích, phần này chỉ xem xét đến mạng truyền thẳng 3 lớp (feed forward 3-layered neural network), ký hiệu NN, là mạng truyền thẳng có cấu trúc chỉ có 1 lớp ẩn (hidden layer). Và một mạng nơ-ron truyền thẳng 3-lớp được tham số hóa bằng 2 ma-trận

$$NN \equiv < A = [a_{ij}] \in \mathbb{R}^{(n+1)\times J}; B = [b_{jk}] \in \mathbb{R}^{(J+1)\times m} >,$$

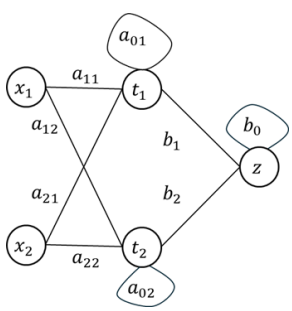
với

. $a_{0,j}, j = 1, \dots, J$ là véc-tơ ngưỡng (bias) của các nơ-ron ẩn

. $b_{0,k}, k = 1, \dots, m$ là véc-tơ ngưỡng của các nơ-ron xuất.

. J là số nơ-ron ẩn. Giá trị cụ thể của J do người thiết kế mạng NN quyết định.

Chính vì có thêm các nơ-ron ẩn mà NN có khả năng xấp xỉ một hàm phi tuyến. Chẳng hạn, hàm phi tuyến xor có thể được mô hình bằng NN có 2 nơ-ron ẩn như hình vẽ.



Đạo hàm hàm lỗi

Từ công thức học

$\boldsymbol{w}^* = \boldsymbol{w}^* - \alpha\nabla$ hay

$w = w + \alpha\Delta$,(*)

với $\boldsymbol{w}$ ký hiệu ma-trận trọng số A hay B; và w ký hiệu 1 trọng số thành phần.

Công thức cập nhật trong số thành phần (*) phụ thược giá trị đạo hàm riêng Δ, được xác định theo các công thức sau.

. Trước hết, xét trọng số đầu ra $[b_{jk}]$. Để đơn giản công thức, ta sẽ chỉ sử dụng các chỉ số khi cần làm rõ. Sử dụng quy tắc chuỗi đạo hàm hàm hợp.

$\frac{\partial E}{\partial b} = \frac{\partial E}{\partial z}\frac{\partial z}{\partial v}\frac{\partial v}{\partial b}$.

Từ $E = \frac{1}{2}\ (z - y)^2$, suy ra

$\frac{\partial E}{\partial z} = z - y$.

Do $z = g(v)$, ta có

$\frac{\partial z}{\partial v} = g(v)(1 - g(v)) = z(1 - z)$.

Cuối cùng, từ $v = b_0 + t_1 b_1 + \cdots + t_j b_j + \cdots + t_J b_J$, thì

$\frac{\partial v}{\partial b_j} = \begin{cases} 1, & j = 0 \\ t_j, & j \geq 0 \end{cases}$.

Đặt $p = \frac{\partial E}{\partial z}\frac{\partial z}{\partial v} = (z - y)z(1 - y)$,

$\frac{\partial E}{\partial b_j} = \begin{cases} p, & j = 0 \\ pt_j, & j > 0 \end{cases}$ với mọi $j = 0,1, \dots, J$.

. Bây giờ, xét các trong số tầng ẩn $[a_{ij}]$. Vẫn sử dụng quy tắc chuỗi đạo hàm hàm hợp.

$\frac{\partial E}{\partial a} = \frac{\partial E}{\partial t}\frac{\partial t}{\partial u}\frac{\partial u}{\partial a}$.

Xét thừa số $\frac{\partial E}{\partial t}$

$\frac{\partial E}{\partial t} = \Sigma_{k=1}^{m} \frac{\partial E}{\partial z_k}\frac{\partial z_k}{\partial v_k}\frac{\partial v_k}{\partial t} = \Sigma_{k=1}^{m} p_k \frac{\partial v_k}{\partial t}$,

trong đó, ký hiệu $b_k \equiv b_{jk}$

$\frac{\partial v_k}{\partial t} = b_k$, do $v_k = b_0 + b_1 t_1 + \cdots + b_J t_J$.

Nên

$\frac{\partial E}{\partial t} = \Sigma_{k=1}^{m} p_k b_k.$

Do $t = g(u)$, nên

$\frac{\partial t}{\partial u} = t(1-t).$

Đặt $q = \frac{\partial E}{\partial t}\frac{\partial t}{\partial u}$ hay

$q = (\Sigma_{k=1}^{m} p_k b_k) t(1-t),$

ta xét

$\frac{\partial u}{\partial a_i}, i = 0,1, \dots, n.$

Do $u = a_0 + x_1 a_1 + \cdots + x_i a_i + \cdots + x_n a_n$, nên

$\frac{\partial u}{\partial a_i} = \begin{cases} 1, & i = 0 \\ x_i, & i \geq 0 \end{cases}.$

Vậy

$\frac{\partial E}{\partial a_i} = \begin{cases} q, & i = 0 \\ q x_i, & i > 0 \end{cases}.$

Học thích nghi

Trong công thức học,

$w_i^* = w_i^* - \alpha \Delta_i = w + C.$

$C = -\alpha \Delta$

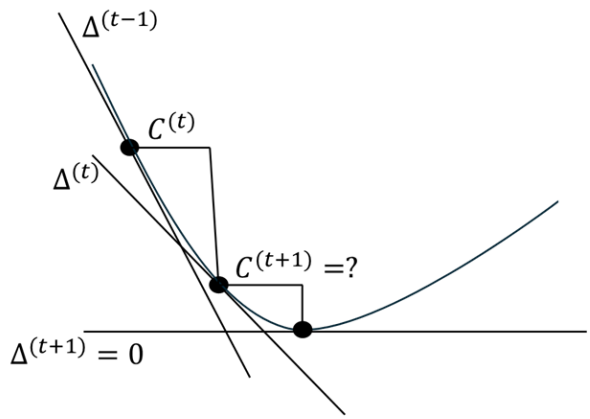
được gọi là hệ số học.

Hệ số học α là hằng số, cố định với mọi bước học, và như thế, có thể ảnh hưởng đến tốc độ hội tụ của thuật giải.

Máy học, thực chất là tối ưu hóa hàm mất mát, hàm lỗi hay hàm sai số đạt giá trị nhỏ nhất có thể. Trong phần cuối chương trước, bài toán tối ưu được giải bằng kỹ thuật xấp xỉ hàm mục tiêu bằng chuỗi các parabol. Ta cũng sử dụng kỹ thuật parabol để xấp xỉ đạo hàm riêng

Δ_i trong véc-tơ đạo hàm. Phương pháp này, được gọi là QuickProp – Quick Propagation, được đề xuất bởi Scott Fahlman, năm 1989[1].

Ta biết, qua 3 điểm, hay tổng quát, có 3 thông tin, xác định 1 parabol. Ở đây, thay vì lấy thông tin 3 điểm phân biệt, ta sẽ sử dụng các thông tin đạo hàm tại các bước trước. Hình vẽ sau minh họa quá trình tính các thành phần của tiến trình xấp xỉ bằng parabol.



Gọi

$\Delta^{(t-1)}, \Delta^{(t)}$ lần lượt là đạo hàm riêng phần tại bước lặp trước, và bước hiện tại.

$C^{(t-1)}, C^{(t)}$ là biến thiên trọng thích nghi đã biết tại bước lặp trước, và bước kế cần tính.

Mục đích của ta là tìm giá trị của trọng số mà tại đó sai số đạt giá trị nhỏ nhất, chính là điểm cực tiểu của parabol. Tại điểm cực trị, đạo hàm hàm lỗi bằng $\Delta^{(t+1)} = 0$. Do đạo hàm hiện hành là Δ^{t}. Độ lệch đạo hàm sẽ là $\Delta^{(t-1)} - \Delta^{(t)}$. Như vậy, tỷ lệ giảm sẽ là $\frac{\Delta^{(t)} - \Delta^{(t+1)}}{\Delta^{(t-1)} - \Delta^{(t)}}$. Tương ứng với tỷ lệ biến thiên trọng là $\frac{C^{(t)}}{C^{(t-1)}}$:

$$\frac{C^{(t)}}{C^{(t-1)}} = \frac{\Delta^{(t)} - \Delta^{(t+1)}}{\Delta^{(t-1)} - \Delta^{(t)}},$$

Hay

$$C^{(t)} = \frac{\Delta^{(t)} - \Delta^{(t+1)}}{\Delta^{(t-1)} - \Delta^{(t)}} C^{(t-1)} \ (*)$$

C trong công thức (*) được gọi là biến thiên trọng thích nghi.

[1] S. E. Fahlman, An empirical study of learning speed in back-propagation networks (1988), Carnegie Mellon University, Computer Science Department.

Lưu ý là khi $\Delta^{(t-1)} = \Delta^{(t)}$, vế phải (*) không xác định. Khi ấy, biến thiên trọng được tính theo công thức chuẩn

$C^{(t-1)} = -\alpha\Delta$.

MẠNG NƠ-RON NỬA TĨNH

Về mặt hình thức, mạng nơ-ron đa-lớp là một hàm hợp

$\boldsymbol{y} = f_{\boldsymbol{w}}(\boldsymbol{x}) = f_P(\dots f_2(f_1(\boldsymbol{x}, \boldsymbol{\Theta}_1), \boldsymbol{\Theta}_2) \dots, \boldsymbol{\Theta}_P)$.

Trong trường hợp mạng 3-lớp ta đã khảo sát ở trên,

$\boldsymbol{y} = f_2(f_1(\boldsymbol{x}), A), B)$,

với

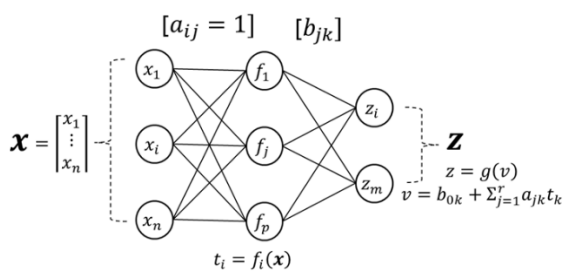
. $f_1(\boldsymbol{x}, A) = \boldsymbol{t} = G(A \cdot \boldsymbol{x})$, và

. $f_2(\boldsymbol{t}, B) = \boldsymbol{z} = G(B \cdot \boldsymbol{t})$,

trong đó, nếu $\boldsymbol{v} \in \mathbb{R}^k$, thì $G(\boldsymbol{v}) = (g(v_1), \dots, g(v_k))$ và g là hàm sigmoid.

Mạng nửa tĩnh

Mạng nơ-ron nửa tĩnh (semi-static neural network), hay gọi tắt là mạng nửa tính (semi-s**T**atic **N**eural **N**etwork), ký hiệu T-NN, là một mạng nơ-ron có cấu trúc "lưng chừng" (halfway) giữa perceptron và 3-layered neural network, và là một mạng 3-lớp như hình vẽ, trong đó các trọng số của lớp ẩn được xác định là hằng số và bằng 1, $A = \left[a_{ij} = 1\right]_{i,j}$, và các trong số của lớp xuất được xác định bằng phương pháp perceptron.



Về mặt hình thức thì T-NN là sự mở rộng kỹ thuật thử dữ liệu hồi quy tuyến tính (linear regression-based data fitting) kết hợp với perceptron. Dữ liệu đầu vào $\boldsymbol{x} \in \mathbb{R}^n$, trước hết được phi-tuyến hóa qua lớp phi tuyến hóa T, $\boldsymbol{t} = T(\boldsymbol{x}) = (f_1(\boldsymbol{x}), \dots, f_p(\boldsymbol{x}))$, sau đó được tính toán bằng perceptron để cho kết quả đầu ra, $z = G(B \cdot \boldsymbol{t})$. Trong đó,

$f_j: \mathbb{R}^n \to \mathbb{R}, j = 1, \dots, p$,

là các hàm cơ sở được xác định trước, như trong phương pháp thử dữ liệu ở trên. Và $G(.)$ là

$G(\boldsymbol{B} \cdot \boldsymbol{t})$

được định nghĩa như trong mục trước.

Máy học mạng nửa tĩnh

Một mạng nửa tĩnh, $T_NN: \mathbb{R}^n \to \mathbb{R}^m$ được xây dựng dựa trên tập mẫu $\Omega = \left\{(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)}) \in \mathbb{R}^{n \times m}\right\}_i$ gồm 3 bước cơ bản:

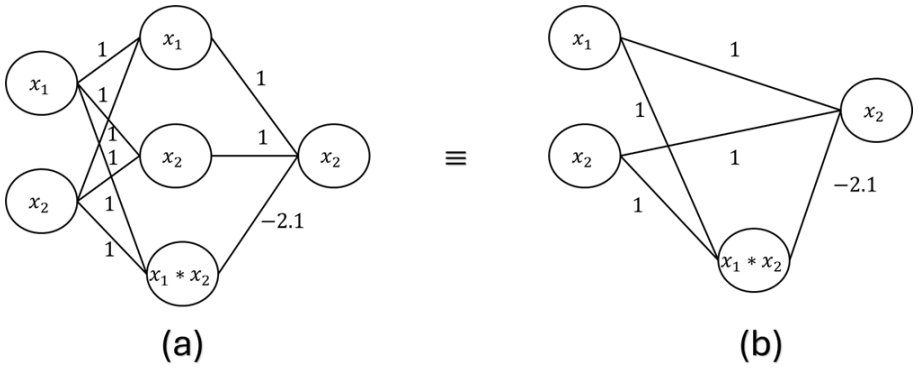
Bước 1: phân tích dữ liệu và định nghĩa các hàm cơ sở

$f_j: \mathbb{R}^n \to \mathbb{R}, j = 1, \dots, p.$

Bước 2: học perceptron trên tập dữ liệu biến đổi

$\Omega \leftarrow T(\Omega) = \left\{(f_1(x^{(i)}, y^{(i)}), \dots, f_p(x^{(i)}, y^{(i)})) \in \mathbb{R}^{p \times m}\right\}_i.$

Hình sau minh họa một T-NN học hàm XoR.



Hai mạng (a) và (b) là cùng một T-NN học hàm XoR. Cấu trúc mạng (b) minh họa rõ hơn về tính chất nửa tĩnh của T-NN là kết hợp của data fitting với perceptron.

HỌC SÂU

Khái niệm học sâu

Học sâu và trí tuệ nhân tạo

Học là quá trình tích lũy tri thức, kinh nghiệm từ những thông tin nhận được. Trong học từ dữ liệu, thông tin được chắt lọc từ dữ liệu và được tổng hợp cô đọng thành kiến thức của hệ thống. Học từ dữ liệu có thể ví như học sinh học bài của môn học, với bài học là các mẫu trong tập dữ liệu được cho trong lãnh vực đó. Học thuộc lòng (learn by heart) là khi

học sinh có thể trả lời trơn tru, hay va vấp rất ít, những bài đã được học. Nhưng mục tiêu của học không chỉ giới hạn trong việc trả bài như thế. Mục tiêu của học là hiểu biết sâu xa về đối tượng môn học nghiên cứu. Nghĩa là mô hình có thể dự đoán được cả những gì không được học. Nếu xem học thuộc lòng là học nông (shallow learning), thì học hiểu bản chất đối tượng là học sâu (deep learning).

Học thuộc lòng nhiều bài học để có thể trả lời được câu hỏi cụ thể có thể được xem là biết rộng hiểu sâu. Như vậy, có thể xấp xỉ

Học Sâu = Học Nông + Dữ liệu lớn.

Triết lý "lấy cần cù bù thông minh" này rất phù hợp với Khoa học Máy tính: dùng sức mạnh không mệt mỏi của máy tính làm cho máy tính thông minh hơn để máy tính có trí tuệ hay để có "trí tuệ nhân tạo".

Bằng kỹ thuật số hóa dữ liệu, khai niệm học sâu có thể được cài đặt bằng các mạng nơ-ron học sâu.

Hiệu suất mô hình

Mạng 3-lớp, số số nơ-ron ẩn thích hợp, có thể học được hầu hết tập mẫu Ω được cho với sai số $\varepsilon > 0$ nhỏ bất kỳ cho trước. Tương tự, mạng nửa tĩnh T-NN, với các hàm cơ sở thích hợp cũng có thể mô hình hóa xấp xỉ được hầu hết tập mẫu Ω được cho. Từ đây, ta sẽ gọi chung là mạng đa lớp, bao gồm cả mạng nửa tĩnh (T-NN), mạng 1 hay nhiều lớp nơ-ron ẩn.

Khả năng xấp xỉ của mạng 1 lớp ẩn là do các nơ-ron ẩn với hàm sigmoid g phi tuyến hóa tổng dữ liệu đầu vào, $g(A \cdot \boldsymbol{x}) = g(u)$, và như thế các nơ-ron đầu ra của mạng có dạng bán phi tuyến, $g(B \cdot \boldsymbol{u})$.

Thực nghiệm cho thấy, số nơ-ron ẩn càng nhiều thì khả năng xấp xỉ càng khớp với tập dữ liệu (data fitting) trong tập mẫu Ω được học.

Nhiệm vụ của hầu hết các phương pháp mô hình hóa là xây dựng được mô hình có hiệu năng sử dụng (performance) cao, nghĩa là mô hình xây dựng được không chỉ tính chính xác những mẫu đã được học trong tập mẫu (fitting), mà quan trọng hơn, mô hình phải có khả năng tính chính xác các mẫu không được học. Tính chất này được gọi là tính tổng quát hóa (generalization) của mô hình. Hiệu năng của mô hình được đo dựa trên cả 2 tính chất: khớp và tổng quát.

Tính khớp dữ liệu được xác định qua quá trình huấn luyện (training) mô hình, trong khi tính tổng quát hóa được xác định khi sử dụng (using) mô hình, nghĩa là khi mô hình đã

được xây dựng. Hiệu năng của mô hình, tính khớp và tính tổng quát thường không đồng biến với nhau: tính khớp có tính chất giảm đơn điệu (monotone), trong khi tính tổng quát của mô hình được luyện đó thì không. Thường thì mô hình càng khớp càng tổng quát hóa kém. Do tính đơn điệu giảm, số bước lặp càng lớn, độ khớp càng cao. Độ khớp cao cũng có thể đạt được bằng cách thêm số nơ-ron ẩn. Như vậy, để giảm độ khớp, có thể xem đồng nghĩa với tăng độ tổng quát, có thể sử dụng ít nơ-ron ẩn, hoặc số bước học ít; hoặc cả hai.

Vấn đề đang ở chỗ, với cấu hình mạng 3-lớp cố định trước số nơ-ron ẩn, chỉ có thể đo được độ tổng quát hóa khi mô hình đã được huấn luyện xong, nghĩa là không thể thay đổi được số bước học đã qua. Một giải pháp dung hòa là phân hoạch tập dữ liệu mẫu thành 2 tập tách biệt nhau:

$\Omega = \Omega_{\text{Train}} \cup \Omega_{\text{Validation}}$,

tập Ω_{Train} được dùng để huấn luyện mô hình, còn tập $\Omega_{\text{Validation}}$ được dùng để dừng tiến trình học.

Theo đó, tại mỗi giai đoạn (epoch) huấn luyện gồm một số bước lặp thích hợp, mạng được học trên tập Ω_{Train}, nghĩa là được đánh giá về độ khớp; và cuối epoch, mạng được đánh giá độ tổng quát hóa dựa trên tập $\Omega_{\text{Validation}}$. Quá trình huấn luyện sẽ dừng khi độ tổng quát hóa không giảm nữa.

Thực ra vấn đề không nằm ở chỗ thuật giải, vấn đề nằm ở chỗ chọn cho được số nơ-ron hay số bước lặp thích hợp. Khi tập mẫu nhỏ, việc lại chia nhỏ tập mẫu thành 2 tập tách biệt có khi dẫn đến thất bại trong cả 2 nhiệm vụ: khớp dữ liệu lẫn tổng quát hóa.

Tóm lại, vấn đề của máy học–mạng nơ-ron, hay tổng quát hơn là vấn đề của máy học, để đạt được mô hình có hiệu suất cao cần dữ liệu lớn (big data), vì bản chất các phương pháp học này là học-từ-dữ-liệu (learning from data or learning from observations).

Mạng nơ-ron học sâu

Mạng nơ-ron học sâu, hay đơn giản là mạng học sâu (deep learning network), về cấu trúc vật lý là mạng có chiều sâu gồm nhiều nơ-ron ẩn được phân bố vào chỉ 1 tầng hay nhiều tầng của mạng đa tầng. Về logic, là mạng có khả năng biểu diễn sâu xa về lãnh vực nó đã học qua tập dữ liệu lớn (big data).

Từ khả năng một mạng đa-lớp có "đủ" số nơ-ron ẩn có thể khớp (fit) được mọi tập dữ liệu được cho. Hệ quả là mô hình khớp với tập huấn luyện rất lớn sẽ tổng quát hóa (generalize) tốt. Một cách trực quan, để học một tập mẫu lớn cần số nơ-ron ẩn lớn.

Bên cạnh kích cỡ mạng lớn, nhiều lớp ẩn với nhiều nơ-ron ẩn thuộc từng lớp, khái niệm học sâu (deep learning) còn ám chỉ hệ thống sử dụng cả tri thức thuộc lãnh vực vào biểu diễn dữ liệu lẫn vào tiến trình học. Các mạng học sâu phổ biến là CNN, RNN, và SLTM.

CNN – mạng tích chập

Mạng nơ ron tích chập – CNN (Convolution Neural Network) là mạng học sâu sử dụng các phép tích chập để xử lý các tín hiệu số[2], đặc biệt tín hiệu có cấu trúc 2 chiều, vì thế CNN rất thích hợp cho các bài toán liên quan đến dữ liệu có cấu trúc 2-chiều như ảnh chẳng hạn.

Ý tưởng chính của CNN là sử dụng các lớp chập (convolution) để trích xuất các đặc trưng cục bộ từ dữ liệu đầu vào của đối tượng, sau đó sử dụng các lớp gộp (pooling) để giảm kích thước và làm nổi bật các đặc trưng quan trọng của đối tượng. Kỹ thuật cơ bản là áp dụng bộ lọc lên dữ liệu đầu vào để tạo ra bản đồ đặc trưng, sau đó giảm kích thước bản đồ này thông qua lớp gộp.

Hoạt động và các thành phần chính của CNN:

- **Lớp chập (Convolutional Layer).** Trong lớp chập, các bộ lọc (filter) được cho trượt trên toàn bộ ảnh đối tượng và trả về bản đồ đặc trưng (feature map) biểu diễn mức độ xuất hiện các đặc trưng lọc được trên ảnh. Mỗi bộ lọc được thiết kế để nhận biết một loại đặc trưng cụ thể, như cạnh, góc hoặc các vân ảnh.
- **Hàm kích hoạt (Activation Function).** Sau đó, một hàm kích hoạt như phi tuyến được áp dụng trên kết quả chập. Điều này giúp mạng có khả năng học các mối quan hệ phức tạp hơn trong dữ liệu. Hàm kích hoạt ReLU thường được áp dụng như hàm kích hoạt.
- **Lớp gộp (Pooling Layer).** Lớp gộp nhằm giảm kích thước không gian của bản đồ đặc trưng. Gộp có thể là lấy giá trị lớn nhất (max pooling), hay giá trịnh trung bình (average pooling) của một vùng. Cách làm này nhằm giảm số lượng tham số và do đó giảm độ quá khớp (over fitting).
- **Lớp liên kết đủ (Fully Connected Layer).** Sau khi qua nhiều lớp chập và lớp gộp, các bản đồ đặc trưng được làm phẳng (flatten) và truyền cho lớp liên kết đủ để tổng hợp tất cả các đặc trưng đã trích xuất và thực hiện phân loại cuối.
- **Lớp đầu ra (Output Layer).** Tại lớp cuối cùng này, hàm xác suất được sử dụng hàm để tính xác suất cho mỗi lớp đầu ra, từ đó có thể xác định được đối tượng hoặc nhãn của dữ liệu đầu vào. Hàm xác suất phổ biến là hàm Softmax.

---

[2] Isidore Isaac Hirschman, David Vernon Widder. The Convolution Transform. Literary Licensing, LLC, April 7, 2012 (2012).

RNN – mạng hồi tiếp

Mạng hồi tiếp – RNN (Recurrent Neural network) là một loại mạng học sâu khác, được thiết kế chuyên xử lý dữ liệu tuần tự (sequential data), là dữ liệu có quan hệ nội tại nhau theo thời gian hoặc trình tự xuất hiện, như dữ liệu chuỗi thời gian hay thời tiết) hay dữ liệu ngôn ngữ tự nhiên chẳng hạn.

Ý tưởng cốt lõi của RNN sử dụng một vùng nhớ đệm để lưu trữ thông tin từ các bước trước đó. Trong RNN, dữ liệu đầu vào và đầu ra được kết hợp với ngữ cảnh của chuỗi dữ liệu để đưa ra dự đoán.

Hoạt động và các thành phần chính của RNN:

- **Trạng thái ẩn (Hidden state).** Là "trạng thái bộ nhớ tạm", lưu giữ thông tin đã được xử lý ở các bước trước trong chuỗi. Khi có một đầu vào mới, RNN sẽ cập nhật trạng thái ẩn này để ghi nhớ ngữ cảnh hiện hành.

- **Chia sẻ trọng số (Weight sharing).** RNN sử dụng cùng tập các trọng số cho mỗi bước thời gian của chuỗi đầu vào, giúp mạng học được các đặc trưng chung của chuỗi mà không cần phải có một tập hợp tham số khác nhau cho từng vị trí trong chuỗi.

- **Nguyên lý hồi tiếp (Recurrent)**. Là nguyên lý cốt lõi của RNN. Thông tin bước trước được đưa ngược trở lại vào mạng để xử lý đầu vào hiện hành. Nguyên lý hồi tiếp được cài đặt qua công thức lặp hồi tiếp:
  $h_t = \sigma(\boldsymbol{w}_{hh} h_{t-1} + \boldsymbol{w}_{xh} x_t + \Delta_h)$,
  trong đó,
  . $\boldsymbol{w}_{hh}, \boldsymbol{w}_{xh}$ lần lượt là các ma-trận trọng số.
  . Δ_h là véc-tơ độ lệch.
  . σ là hàm kích hoạt phi tuyến. Thường là hàm $tanh$ được xác định như $\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$.

RNN gặp 2 vấn đề trong quá trình huấn luyện: (1) khi xử lý chuỗi dài, đạo hàm (gradient) có thể trở nên rất nhỏ (triệt tiêu) hoặc rất lớn (bùng nổ), khiến việc học các phụ thuộc dài hạn trở nên khó khăn. (2) có xu hướng "quên" các thông tin đã xuất hiện từ lâu, làm giảm hiệu quả khi xử lý các chuỗi rất dài. LSTM được phát triển để giải quyết các vấn đề này.

LSTM – Ký ức ngắn-dài hạn

Ký ức ngắn-dài hạn – LSTM (Long-Short Term Memory) là một dạng RNN, được thiết kế để giải quyết các vấn đề mà RNN gặp phải khi huấn luyện trên các chuỗi dài. LSTM sử dụng các ô nhớ và 3 loại cổng để chọn đọc, ghi và xóa thông tin giúp cải thiện khả năng học và nhớ trong các chuỗi dài.

Hoạt động và các thành phần chính của LSTM:

- **Cổng quên (Forget Gate).** Quyết định thông tin nào từ trạng thái ô trước đó cần được giữ lại hay loại bỏ. Nhận đầu vào hiện tại x_t và trạng thái ẩn h_{t-1} trước đó, và trả về véc-tơ các giá trị thuộc $[0, 1]$: gần 0 có thể quên; và nhớ nếu gần 1. Công thức tính véc-tơ kết quả sử dụng hàm sigmoid: $f_t = \sigma(\boldsymbol{w}_f \cdot [h_{t-1}, x_t] + \nabla_f)$.
- **Cổng nhập (Input Gate).** Quyết định thông tin mới nào sẽ được thêm vào trạng thái ô. Cổng này có 2 thành phần
  . Lớp sigmoid quyết định giá trị nào cần cập nhật: $i_t = \sigma(\boldsymbol{w}_i \cdot [h_{t-1}, x_t] + \nabla_i)$.
  . Lớp tanh tạo ra véc-tơ úng viên mới để thêm vào trạng thái ô: $\tilde{C}_C = \tanh(\boldsymbol{w}_i \cdot [h_{t-1}, x_t] + \nabla_C)$.
- **Cổng xuất (Output Gate).** Điều chỉnh thông tin nào trong trạng thái ô hiện hành sẽ thành trạng thái ẩn hiện hành. Cổng này gồm 2 thành phần
  . Lớp sigmoid quyết định phần nào trạng thái ô sẽ được xuất ra: $o_t = \sigma(\boldsymbol{w}_o \cdot [h_{t-1}, x_t] + \nabla_o)$.
  . Lớp tanh xử lý trạng thái ô hiện hành, C_t, rồi nhân với kết quả o_t từ lớp sigmoid: $h_t = o_t \cdot \tanh(C_t)$.